

آزمایشگاه طراحی سیستم های دیجیتال

گروه آموزشی مهندسی کامپیوتر دانشگاه صنعتی شریف



دانشکده مهندسی کامپیوتر دانشگاه صنعتی شریف

بهار ۱۴۰۵

استاد: دکتر انصاری

اعضای گروه:

کیمیا قدیر ۴۰۲۱۰۶۳۲۶

کیمیا رضائی ۴۰۲۱۰۶۰۴۸

ملینا طاهری ۴۰۲۱۰۶۱۶۷

گزارشکار آزمایش هفتم

عنوان آزمایش:

طراحی فرستنده و گیرنده UART با استفاده از زبان Verilog

هدف آزمایش:

هدف از انجام این آزمایش طراحی یک واحد UART یا Universal Asynchronous Receiver Transmitter است. در این آزمایش یک کد ۷ بیتی ASCII به صورت سریال ارسال می شود. در ابتدا یک بیت شروع یا Start ارسال می شود، سپس یک بیت توازن یا Parity و بعد از آن ۷ بیت داده به ترتیب روی خط سریال قرار می گیرند. در انتها نیز یک بیت خاتمه یا Stop ارسال می شود. بنابراین در هر انتقال، مجموعاً ۱۰ بیت روی خط ارسال می شود. در قسمت گیرنده نیز پس از تشخیص بیت شروع، بیت توازن و ۷ بیت داده به صورت سریال دریافت می شوند و در یک ثبات ۸ بیتی ذخیره می گردند. سپس با بررسی بیت توازن، درستی یا نادرستی داده دریافتی مشخص می شود. در نهایت با اتصال خروجی فرستنده یک UART به ورودی گیرنده UART دیگر، صحت انتقال داده بررسی می شود.

شرح کلی آزمایش:

توضیح کلی طراحی:

در این آزمایش برای طراحی UART از سه ماژول اصلی استفاده شده است. ماژول sender وظیفه ارسال سریال داده، ماژول receiver وظیفه دریافت سریال داده و بررسی بیت توازن، و ماژول uart وظیفه اتصال این دو بخش به یکدیگر را دارد. در تست پنج نیز دو واحد UART به هم متصل شده اند تا صحت انتقال داده بررسی شود. در ابتدا در ماژول ها دو پارامتر Baudrate و Clockrate تعریف شده اند.

قطعه کد:

```
module sender #(
    parameter Baudrate = 115200,
    parameter Clockrate = 50000000
) (
```

پارامتر Baudrate نرخ ارسال داده و Clockrate فرکانس کلاک مدار را مشخص می کند. با استفاده از این دو مقدار، تعداد سیکل های کلاک لازم برای ارسال یا دریافت هر بیت محاسبه می شود.

```
localparam integer CLKS_PER_BIT = Clockrate / Baudrate;
```

این مقدار مشخص می کند که هر بیت باید چند سیکل کلاک روی خط سریال باقی بماند. برای مثال در تست بنچ اگر Clockrate برابر ۱۶ و Baudrate برابر ۴ باشد، هر بیت به مدت ۴ سیکل کلاک ارسال یا دریافت می شود. در ماژول فرستنده برای کنترل مراحل ارسال از ماشین حالت استفاده شده است.

```
16      localparam IDLE   = 3'd0;
17      localparam START  = 3'd1;
18      localparam PARITY = 3'd2;
19      localparam DATA  = 3'd3;
20      localparam STOP   = 3'd4;
```

این حالت ها مراحل مختلف ارسال را مشخص می کنند. در حالت IDLE خط سریال در مقدار ۱ قرار دارد. در حالت START بیت شروع با مقدار ۰ ارسال می شود. سپس در حالت PARITY بیت توازن ارسال می گردد. بعد از آن در حالت DATA هفت بیت داده به صورت سریال ارسال می شوند و در پایان در حالت STOP بیت خاتمه با مقدار ۱ روی خط قرار می گیرد.

```
if (start) begin
    data_reg    <= data;
    parity_reg  <= ^data;    // even parity
    serial_out  <= 1'b0;    // start bit
    busy       <= 1'b1;
    delay      <= CLKS_PER_BIT - 1;
    state      <= START;
end
```

در این بخش، با فعال شدن سیگنال start، داده ورودی در رجیستر data_reg ذخیره می شود. سپس بیت parity با XOR کردن بیت های داده محاسبه می شود. بعد از آن خروجی serial_out برابر ۰ می شود تا بیت شروع ارسال گردد و فرستنده وارد حالت START می شود.

```
START: begin
    if (delay == 0) begin
        serial_out <= parity_reg;    // parity bit
        delay     <= CLKS_PER_BIT - 1;
        state     <= PARITY;
    end else begin
        delay <= delay - 1;
    end
```

در این قسمت بیت شروع به مدت زمان یک بیت روی خط باقی می ماند. بعد از تمام شدن تاخیر، بیت parity روی خروجی قرار می گیرد و مدار وارد حالت PARITY می شود.

```

PARITY: begin
    if (delay == 0) begin
        serial_out <= data_reg[0]; // first data bit
        bit_counter <= 0;
        delay <= CLKS_PER_BIT - 1;
        state <= DATA;
    end else begin
        delay <= delay - 1;
    end
end
end

```

پس از ارسال بیت parity، اولین بیت داده روی خروجی سریال قرار می گیرد. در این طراحی داده ها از بیت کم ارزش تر به بیت پر ارزش تر ارسال می شوند.

```

DATA: begin
    if (delay == 0) begin
        if (bit_counter == 6) begin
            serial_out <= 1'b1; // stop bit
            delay <= CLKS_PER_BIT - 1;
            state <= STOP;
        end else begin
            bit_counter <= bit_counter + 1;
            serial_out <= data_reg[bit_counter + 1];
            delay <= CLKS_PER_BIT - 1;
        end
    end else begin
        delay <= delay - 1;
    end
end
end

```

در حالت DATA هفت بیت داده یکی یکی ارسال می شوند. شمارنده bit_counter تعداد بیت های ارسال شده را مشخص می کند. وقتی شمارنده به ۶ برسد، یعنی هر ۷ بیت داده ارسال شده اند. سپس خروجی برابر ۱ می شود و مدار وارد حالت STOP می گردد.

```

STOP: begin
    if (delay == 0) begin
        serial_out <= 1'b1;
        busy <= 1'b0;
        done <= 1'b1;
        state <= IDLE;
    end else begin
        delay <= delay - 1;
    end
end
end

```

در این حالت بیت توقف به مدت یک بیت روی خط قرار می گیرد. بعد از پایان این زمان، سیگنال busy صفر و done یک می شود و فرستنده دوباره به حالت IDLE برمی گردد.

در ماژول receiver نیز از ماشین حالت مشابه استفاده شده است. تفاوت اصلی این است که گیرنده باید ابتدا بیت شروع را تشخیص دهد و سپس بیت های بعدی را در زمان مناسب نمونه برداری کند.

```
localparam integer CLKS_PER_BIT      = Clockrate / Baudrate;  
localparam integer CLKS_PER_HALF_BIT = CLKS_PER_BIT / 2;
```

این مقدار برای نمونه برداری از وسط بیت استفاده می شود. وقتی گیرنده صفر شدن خط سریال را تشخیص می دهد، ابتدا نصف زمان یک بیت صبر می کند تا به وسط بیت شروع برسد و مطمئن شود که بیت شروع واقعی است.

```
IDLE: begin  
    delay      <= 0;  
    bit_counter <= 0;  
    parity_error <= 1'b0;  
  
    if (serial_in == 1'b0) begin  
        delay <= CLKS_PER_HALF_BIT - 1;  
        state <= START;  
    end  
end
```

در حالت IDLE، گیرنده منتظر می ماند تا خط serial_in صفر شود. صفر شدن خط نشان دهنده شروع فریم است. پس از آن گیرنده وارد حالت START می شود.

```
START: begin  
    if (delay == 0) begin  
        if (serial_in == 1'b0) begin  
            delay <= CLKS_PER_BIT - 1;  
            state <= PARITY;  
        end else begin  
            state <= IDLE;  
        end  
    end else begin  
        delay <= delay - 1;  
    end  
end
```

در این بخش اگر پس از گذشت نصف زمان یک بیت، مقدار serial_in هنوز صفر باشد، بیت شروع معتبر در نظر گرفته می شود و گیرنده برای دریافت بیت parity آماده می شود. اگر مقدار خط دوباره ۱ شده باشد، گیرنده آن را نویز در نظر گرفته و به حالت IDLE برمی گردد.

```
PARITY: begin
    if (delay == 0) begin
        parity_reg <= serial_in;
        parity_bit <= serial_in;
        delay      <= CLKS_PER_BIT - 1;
        state      <= DATA;
    end else begin
        delay <= delay - 1;
    end
end
```

در این حالت بیت parity دریافت و در parity_reg ذخیره می شود. سپس گیرنده وارد حالت DATA می شود تا هفت بیت داده را دریافت کند.

```
DATA: begin
    if (delay == 0) begin
        data_reg[bit_counter] <= serial_in;

        if (bit_counter == 6) begin
            delay <= CLKS_PER_BIT - 1;
            state <= STOP;
        end else begin
            bit_counter <= bit_counter + 1;
            delay      <= CLKS_PER_BIT - 1;
        end
    end else begin
        delay <= delay - 1;
    end
end
```

در این قسمت بیت های داده یکی یکی از ورودی serial_in خوانده شده و در data_reg ذخیره می شوند. پس از دریافت هفت بیت، گیرنده وارد حالت STOP می شود.

```

STOP: begin
    if (delay == 0) begin
        if (serial_in == 1'b1) begin
            data <= data_reg;
            data_with_parity <= {data_reg, parity_reg};

            if ((^data_reg) == parity_reg) begin
                parity_error <= 1'b0;
            end else begin
                parity_error <= 1'b1;
            end

            valid <= 1'b1;
        end else begin
            parity_error <= 1'b1;
        end

        state <= IDLE;
    end else begin
        delay <= delay - 1;
    end
end
end

```

در حالت STOP ابتدا بررسی می شود که بیت توقف برابر ۱ باشد. سپس داده دریافتی در خروجی data قرار می گیرد و ثبات ۸ بیتی data_with_parity ساخته می شود. بعد از آن بیت parity دریافتی با XOR داده دریافت شده مقایسه می شود. اگر برابر باشند، داده صحیح است و parity_error صفر می شود. اگر برابر نباشند، خطای parity اعلام می شود. در پایان valid فعال می شود تا نشان دهد دریافت یک فریم کامل انجام شده است.

در ماژول uart، فرستنده و گیرنده کنار هم قرار داده شده اند.

```

sender #(
    .Baudrate(Baudrate),
    .Clockrate(Clockrate)
) sender1 (
    .rst(rst),
    .clk(clk),
    .start(start),
    .data(data_sender),
    .serial_out(tx),
    .busy(tx_busy),
    .done(tx_done)
);

receiver #(
    .Baudrate(Baudrate),
    .Clockrate(Clockrate)
) receiver1 (
    .rst(rst),
    .clk(clk),
    .serial_in(rx),
    .data(data_receiver),
    .parity_bit(parity_receiver),
    .data_with_parity(data_with_parity),
    .valid(valid),
    .parity_error(parity_error)
);

```

در این بخش خروجی فرستنده به tx وصل شده و ورودی گیرنده از rx گرفته می شود. بنابراین ماژول uart هم قابلیت ارسال و هم قابلیت دریافت دارد.

در تست بنچ برای بررسی عملکرد، دو واحد UART به هم وصل شده اند.

```
assign rx2 = tx1;
```

با این اتصال، خروجی tx واحد اول به ورودی rx واحد دوم وصل می شود. بنابراین داده ای که توسط UART اول ارسال می شود، توسط UART دوم دریافت می گردد.

```
78 task send_and_check;
79     input [6:0] test_data;
80     begin
81         @(posedge clk);
82         data_sender1 <= test_data;
83         start1 <= 1'b1;
84
85         @(posedge clk);
86         start1 <= 1'b0;
87
88         wait(valid2 == 1'b1);
89         @(posedge clk);
90
91         if (parity_error2) begin
92             $display("ERROR: Parity error detected for data = %b", test_data);
93         end else if (data_receiver2 != test_data) begin
94             $display("ERROR: Sent = %b, Received = %b", test_data, data_receiver2);
95         end else begin
96             $display("PASS : Sent = %b, Received = %b, Parity = %b, Register8 = %b",
97                     test_data, data_receiver2, parity_receiver2, data_with_parity2);
98         end
99
100         repeat(5) @(posedge clk);
101     end
102 endtask
```

برای اینکه ارسال و بررسی داده ها چند بار تکرار نشود، از یک task به نام send_and_check استفاده شده است. این task یک داده ۷ بیتی را به عنوان ورودی می گیرد و همان داده را توسط UART اول ارسال می کند.

در ابتدا مقدار ورودی task روی data_sender1 قرار می گیرد و سپس سیگنال start1 برای یک سیکل کلاک فعال می شود. با این کار فرستنده شروع به ارسال فریم می کند. بعد از آن start1 دوباره صفر می شود تا فقط یک بار عملیات ارسال آغاز شود.

سپس تست بنچ منتظر می ماند تا سیگنال valid2 در گیرنده فعال شود. فعال شدن valid2 یعنی UART دوم یک فریم کامل شامل بیت شروع، بیت توازن، ۷ بیت داده و بیت توقف را دریافت کرده است.

بعد از دریافت کامل داده، ابتدا سیگنال parity_error2 بررسی می شود. اگر این سیگنال یک باشد، یعنی خطای توازن رخ داده است. اگر خطای توازن وجود نداشته باشد، مقدار data_receiver2 با داده ارسالی مقایسه می شود. در صورتی که این دو مقدار برابر باشند، پیام PASS چاپ می شود و یعنی انتقال داده درست انجام شده است. اگر برابر نباشند، پیام ERROR چاپ می شود.

در پایان task چند سیکل کلاک صبر می شود تا فرستنده و گیرنده به حالت آماده برگردند و سپس داده بعدی ارسال شود.

```
send_and_check(7'b0000000);  
send_and_check(7'b1111111);  
send_and_check(7'b1010101);  
send_and_check(7'b1100101);  
send_and_check(7'b0010110);  
send_and_check(7'b1000010);  
send_and_check(7'b0000001);  
send_and_check(7'b1000000);
```

در تست پنج چند داده مختلف ارسال می شوند. برای هر داده، ابتدا سیگنال start فعال می شود و سپس تست پنج منتظر فعال شدن valid در گیرنده می ماند. بعد از دریافت کامل داده، مقدار دریافتی با مقدار ارسالی مقایسه می شود. اگر داده ها برابر باشند و parity_error صفر باشد، انتقال داده درست انجام شده است.

```
VSIM 6> run -all  
# PASS : Sent = 0000000, Received = 0000000, Parity = 0, Register8 = 00000000  
# PASS : Sent = 1111111, Received = 1111111, Parity = 1, Register8 = 11111111  
# PASS : Sent = 1010101, Received = 1010101, Parity = 0, Register8 = 10101010  
# PASS : Sent = 1100101, Received = 1100101, Parity = 0, Register8 = 11001010  
# PASS : Sent = 0010110, Received = 0010110, Parity = 1, Register8 = 00101101  
# PASS : Sent = 1000010, Received = 1000010, Parity = 0, Register8 = 10000100  
# PASS : Sent = 0000001, Received = 0000001, Parity = 1, Register8 = 00000011  
# PASS : Sent = 1000000, Received = 1000000, Parity = 1, Register8 = 10000001  
# UART test finished.  
# ** Note: $finish : D:/term/DSDL/7/test_bench.v(130)  
# Time: 811 ns Iteration: 1 Instance: /test_bench
```